

Java Backend Architecture

Interview Series

Chapter 15.3

REST vs gRPC vs GraphQL

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 15.3 – REST vs gRPC vs GraphQL

Introduction

Choosing the right API protocol is a foundational architectural decision. REST dominates public APIs; gRPC excels at high-performance internal service communication; GraphQL solves the over/under-fetching problem for flexible client-driven queries. Each has distinct trade-offs in performance, tooling, type safety, debugging complexity, and ecosystem maturity.

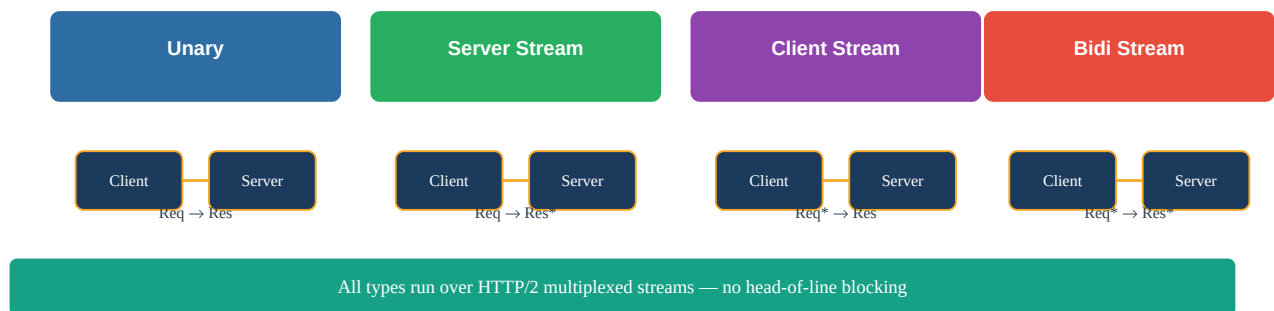
Key Definitions

REST	Representational State Transfer. Resource-based API style using HTTP verbs (GET/POST/PUT/PATCH/DELETE). Stateless, cacheable. Resources identified by URIs. OpenAPI/Swagger for documentation.
gRPC	Google Remote Procedure Call. Uses Protocol Buffers (binary serialization) over HTTP/2. Strongly typed, code-generated stubs, bidirectional streaming. Ideal for internal service communication.
GraphQL	Query language for APIs. Client specifies exactly which fields it needs—no over-fetching (unused fields) or under-fetching (needing multiple requests). Single endpoint, schema-first.
Protocol Buffers	Binary serialization format used by gRPC. Schema defined in .proto files. 3-10x smaller than JSON. Strongly typed. Code generation for client/server stubs across many languages.
HATEOAS	Hypermedia as the Engine of Application State. REST constraint: responses include links to related actions. Enables API discoverability. Rarely implemented in practice.
DataLoader	Batching and caching library for GraphQL (by Facebook). Collects all resolver requests within one event loop tick, executes as a single batch query. Solves the N+1 problem.
N+1 Problem	In GraphQL: fetching a list of N items and then making one additional query per item = N+1 queries. DataLoader batches these into a single query.
OpenAPI	Machine-readable REST API specification (formerly Swagger). Enables code generation, documentation, mocking. Spring Boot generates via springdoc-openapi.
HTTP/2	Binary protocol with multiplexing (multiple streams over one connection), header compression, and server push. Required by gRPC. Eliminates head-of-line blocking.
GraphQL Schema	SDL (Schema Definition Language) defining types, queries, mutations, subscriptions. Acts as contract between client and server. Type-safe and auto-documented.

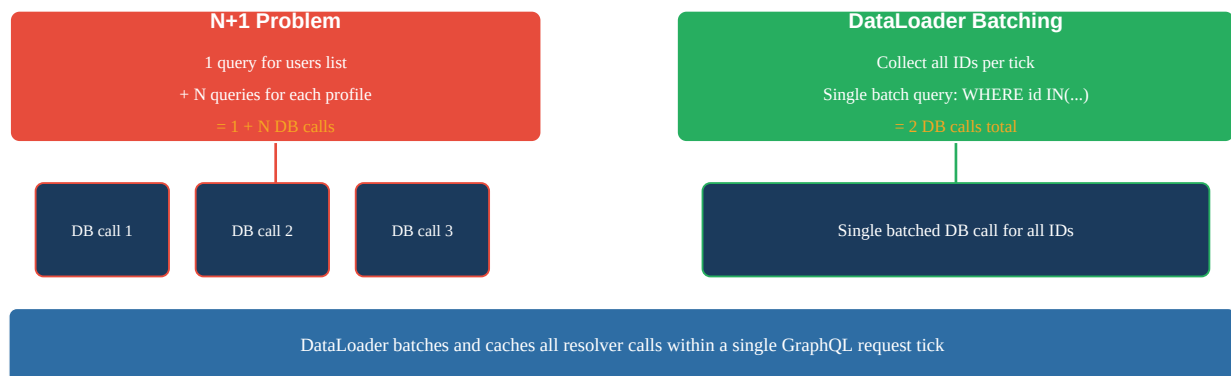
API Protocol Comparison

	REST	gRPC	GraphQL
Protocol	HTTP/1.1+	HTTP/2	HTTP/1.1+
Format	JSON (text)	Protobuf (binary)	JSON
Schema	OpenAPI (opt.)	Protobuf (required)	GraphQL SDL (req.)
Streaming	No (SSE/WS)	Bidi streaming	Subscriptions (WS)
Caching	HTTP cache	Manual	Persisted queries
Best for	Public APIs	Internal svcs	BFF/mobile

gRPC Streaming Types



GraphQL N+1 Problem vs DataLoader Solution



REST – Spring Boot Implementation

```
@RestController @RequestMapping("/api/v1/users") public class UserController {
    @GetMapping("/{id}") public ResponseEntity<UserDto> getUser(@PathVariable Long id) { return
    userService.findById(id) .map(ResponseEntity::ok) .orElse(ResponseEntity.notFound().build());
    } @PostMapping public ResponseEntity<UserDto> createUser(@Valid @RequestBody CreateUserRequest
    req) { UserDto created = userService.create(req); URI location = URI.create("/api/v1/users/" +
    created.id()); return ResponseEntity.created(location).body(created); } @PatchMapping("/{id}")
    public ResponseEntity<UserDto> updateUser(@PathVariable Long id, @RequestBody @Valid
```

```
UpdateUserRequest req) { return ResponseEntity.ok(userService.update(id, req)); } } // OpenAPI
via springdoc-openapi (auto-generated at /v3/api-docs) @OpenAPIDefinition(info = @Info(title =
"User API", version = "v1"))
```

gRPC – Protobuf + Spring Boot

```
// user.proto syntax = "proto3"; package com.example.user; service UserService { rpc GetUser
(GetUserRequest) returns (UserResponse); rpc ListUsers (ListUsersRequest) returns (stream
UserResponse); // Server streaming rpc UpdateUsers (stream UpdateUserRequest) returns
(UserResponse); // Client streaming } message UserResponse { string id = 1; string name = 2;
string email = 3; } // Spring Boot gRPC server (net.devh:grpc-spring-boot-starter) @GrpcService
public class UserGrpcService extends UserServiceGrpc.UserServiceImplBase { @Override public
void getUser(GetUserRequest req, StreamObserver<UserResponse> out) { User user =
userRepository.findById(req.getId()).orElseThrow(); out.onNext(UserResponse.newBuilder()
.setId(user.getId()).setName(user.getName()).build()); out.onCompleted(); } }
```

GraphQL – Spring Boot with DataLoader

```
// schema.graphqls type Query { user(id: ID!): User users: [User!]! } type User { id: ID! name:
String! orders: [Order!]! # N+1 risk without DataLoader } // Spring Boot GraphQL Controller
@Controller public class UserGraphQLController { @QueryMapping public User user(@Argument
String id) { return userService.findById(id); } // DataLoader – batch load orders for all users
in one query @BatchMapping public Map<User, List<Order>> orders(List<User> users) {
List<String> userIds = users.stream().map(User::getId).toList(); Map<String, List<Order>>
ordersByUser = orderService.findByIds(userIds); return
users.stream().collect(Collectors.toMap(u -> u, u -> ordersByUser.getOrDefault(u.getId(),
List.of()))); }
```

Interview Q&A;

Q1. What are the 6 REST constraints?

(1) Client-Server separation, (2) Stateless: no session state on server per request, (3) Cacheable: responses must declare cacheability, (4) Uniform Interface: resource-based URIs + HTTP verbs, (5) Layered System: client can't tell if it's talking to gateway or origin server, (6) Code on Demand (optional): server can send executable code to client.

Q2. What HTTP methods are idempotent?

GET, HEAD, PUT, DELETE, OPTIONS are idempotent: calling them multiple times produces the same result. POST is NOT idempotent—multiple POSTs create multiple resources. PATCH is not idempotent by definition (applying the same patch twice may have different effects), but can be made so with conditional requests.

Q3. What is the difference between PUT and PATCH?

PUT replaces the entire resource with the request body (full update). PATCH applies a partial modification (partial update)—only specified fields are changed. PATCH is better for partial updates as it avoids accidentally overwriting fields the client didn't include. PUT requires the client to send the complete resource.

Q4. Why is gRPC faster than REST/JSON?

Protocol Buffers binary encoding is 3-10x smaller than equivalent JSON. HTTP/2 multiplexing eliminates connection overhead. Code-generated stubs avoid reflection-based serialization. For internal services with high throughput, these gains are significant. Drawback: binary format is not human-readable—harder to debug without tooling.

Q5. What are the gRPC streaming types?

(1) Unary: single request, single response (like HTTP). (2) Server streaming: single request, stream of responses (e.g., live data feed). (3) Client streaming: stream of requests, single response (e.g., file upload). (4) Bidirectional streaming: both client and server send streams simultaneously (e.g., chat).

Q6. What is the N+1 problem in GraphQL?

When querying a list of N users and their orders, a naive resolver calls: 1 query for users + N queries for each user's orders = N+1 queries. With 100 users, that's 101 database queries for one GraphQL request. Solved by DataLoader: collects all order-fetch requests within one tick, executes as single batch query.

Q7. How does DataLoader work?

DataLoader batches all resolver calls that occur within a single event loop tick. Instead of immediately querying the DB, resolvers return a Promise. At the end of the tick, DataLoader collects all requested IDs and fires one batch query: `SELECT * FROM orders WHERE user_id IN (id1, id2, ..., idN)`. Also caches results for duplicate requests.

Q8. When would you choose gRPC over REST?

Choose gRPC for: (1) internal service-to-service communication where performance is critical, (2) bidirectional streaming needs, (3) polyglot environments needing type-safe contracts, (4) high-throughput microservices. Avoid gRPC for public APIs (browser support limited), when HTTP caching is important, or when debugging simplicity matters.

Q9. What is OpenAPI and how does Spring Boot support it?

OpenAPI (formerly Swagger) is a machine-readable REST API spec in YAML/JSON. Documents endpoints, request/response schemas, auth methods. Spring Boot: add `springdoc-openapi-starter-webmvc-ui` dependency. Auto-generates spec at `/v3/api-docs` and UI at `/swagger-ui.html` from controller annotations. Supports code generation for client SDKs via `openapi-generator`.

Q10. What are GraphQL subscriptions?

Real-time data push from server to client over WebSocket. Client subscribes to a subscription type in schema. Server pushes events when data changes. Example: `subscription { orderStatusUpdated(orderId: 123) { status } }`. Spring Boot: `spring-graphql` supports subscriptions via WebSocket or Server-Sent Events.

Q11. What is HATEOAS and is it commonly implemented?

Hypermedia as the Engine of Application State. REST responses include links to related actions. Example: GET `/orders/123` response includes links: `{ 'self': '/orders/123', 'cancel': '/orders/123/cancel', 'payment': '/orders/123/payment' }`. Theoretically enables API discoverability. Rarely implemented in practice—most REST APIs don't include hypermedia links. Spring HATEOAS library supports it.

Q12. How do you handle GraphQL authorization at the field level?

Use custom directives or method-level security. Spring Security + `spring-graphql`: `@PreAuthorize` on `@QueryMapping` methods. For field-level: custom `@SchemaDirective` or check in resolver. Example: `adminEmail` field only returned if user has ADMIN role. `DataFetchingEnvironment` provides access to `SecurityContext`.

Q13. What is Protocol Buffers backwards compatibility?

Protobuf fields are identified by field numbers, not names. Adding new fields with new numbers is safe (old clients ignore unknown fields). Never reuse field numbers—old clients will misinterpret the data. Never remove required fields (proto2). Using optional fields (proto3 default) allows safe removal from schema.

Q14. How do you do versioning in gRPC?

Options: (1) Package versioning: `service com.example.user.v2.UserService` in a new proto package. (2) Run old and new service in parallel, routing clients gradually. (3) Field additions (backward compatible) require no versioning. gRPC's strict schema makes versioning explicit—breaking changes are compile errors for consumers.

Q15. What is REST over-fetching and under-fetching?

Over-fetching: API returns more data than client needs (e.g., `GET /users` returns all 30 fields, client uses 3). Wastes bandwidth. Under-fetching: API doesn't return enough data, requiring multiple requests (`GET /users`, then `GET /users/{id}/orders` for each). GraphQL solves both: client specifies exactly what it needs.

Q16. How does Spring Boot integrate with GraphQL?

`spring-boot-starter-graphql` (GA since Boot 2.7). Loads `.graphqls` schema files from classpath. `@QueryMapping`, `@MutationMapping`, `@SubscriptionMapping` on controller methods. `@BatchMapping` for DataLoader-style batch resolvers. `@SchemaMapping` for field-level resolvers. GraphiQL UI at `/graphql`.

Q17. What is content negotiation in REST?

Client specifies preferred response format via `Accept` header: `Accept: application/json` or `Accept: application/xml`. Server responds with `Content-Type` header indicating actual format. Spring Boot: `@RestController` supports multiple formats via `HttpMessageConverters`. Add `jackson-dataformat-xml` for XML support alongside JSON.

Q18. What are the disadvantages of GraphQL?

(1) Complex queries can be expensive—clients can request deeply nested data causing DB load. (2) Caching is harder—HTTP caching doesn't work with single endpoint POST. (3) N+1 problem requires DataLoader. (4) Learning curve for schema design. (5) File uploads are awkward. (6) Subscriptions add WebSocket complexity. (7) Tooling less mature than REST for security, rate limiting.

Q19. What is the Backend for Frontend (BFF) pattern?

A dedicated API layer per client type (mobile BFF, web BFF). Each BFF tailors responses to its client's specific data needs—aggregates calls to multiple downstream services. Mobile clients get lean responses; web clients get richer data. GraphQL is often used as a BFF—mobile and web clients query only what they need.

Q20. How do you handle authentication in gRPC?

gRPC uses metadata (equivalent to HTTP headers) for auth. Client attaches JWT as metadata: `Metadata.Key JWT_KEY = Metadata.Key.of('Authorization', Metadata.ASCII_STRING_MARSHALLER)`. Server-side interceptor validates JWT. SSL/TLS is standard for gRPC transport security. mTLS (mutual TLS) for service-to-service authentication.

Q21. What is REST idempotency key pattern?

For non-idempotent POST operations (payment, order creation), client generates a unique idempotency key (UUID) and includes it in request header: `Idempotency-Key: uuid`. Server stores key + result. On duplicate request (network retry), returns cached result. Prevents duplicate payments/orders from client retries.

Q22. Compare JSON vs Protocol Buffers encoding efficiency.

JSON user object: `{"id":"123","name":"Alice","age":30}` = ~35 bytes (uncompressed). Same in Protobuf: field numbers + varint encoding ≈ 10-12 bytes. 3-4x size reduction typical. With gzip on JSON, the gap narrows but Protobuf still wins. Binary also parses faster than text parsing. Important for high-throughput internal services.

Q23. What is GraphQL introspection?

GraphQL APIs expose their full schema via introspection queries. Clients can query `__schema` to discover all types, fields, and their documentation. Enables powerful tooling (GraphiQL, code generators). Security: disable introspection in production public APIs to prevent schema enumeration attacks. Spring GraphQL: `spring.graphql.graphiql.enabled=false` in prod.

Q24. How do you implement pagination in REST vs GraphQL?

REST: cursor-based pagination via Link header or `_links`. GET `/users?cursor=abc&limit;=20`. GraphQL: Relay-style cursor pagination (recommended): query `users(first: 20, after: cursor) { edges { node cursor } pageInfo { hasNextPage } }`. Both support offset-based (page/limit) but cursor-based handles real-time data additions better.

Q25. What is a GraphQL mutation?

GraphQL operations that change data (write operations), equivalent to REST POST/PUT/PATCH/DELETE. Mutations are explicitly typed in the schema: `type Mutation { createUser(input: CreateUserInput!): User }`. Unlike queries, mutations execute serially (not in parallel). Best practice: return the modified object from mutations so clients can update their cache.

Q26. How do you implement error handling in GraphQL vs REST?

REST: uses HTTP status codes (400 Bad Request, 404 Not Found, 500 Server Error) + error body. GraphQL: always returns 200 OK. Errors appear in 'errors' array alongside 'data'. Partial success: data contains successful fields, errors contains field-level failures. Spring GraphQL: `DataFetcherExceptionResolverAdapter` for custom error handling.

Q27. When should you use REST vs GraphQL for a public API?

REST: when consumers are diverse (any HTTP client), caching is critical, you need HTTP status semantics, team lacks GraphQL expertise, simple resource-oriented API. GraphQL: when clients have diverse data needs (mobile vs web), over/under-fetching is a real problem, you need a BFF layer, building a developer platform where consumers build their own queries.

Q28. What is gRPC reflection?

gRPC server reflection lets clients discover available services and their proto definitions at runtime. Enables tools like `grpcurl` to call gRPC services without pre-compiled stubs. Enable in Spring Boot: add `ProtoReflectionService` bean. Useful in development; disable in production for security.

Q29. How does HTTP/2 multiplexing benefit gRPC?

HTTP/1.1 requires a new connection per request (or connection reuse with head-of-line blocking). HTTP/2 sends multiple streams over one TCP connection simultaneously. gRPC leverages this: multiple RPC calls interleave over one connection. Eliminates connection setup overhead. Crucial for high-frequency internal service calls.

Q30. What is field-level caching in GraphQL?

HTTP caching doesn't work well with GraphQL (single POST endpoint). Alternatives: (1) Persisted queries: cache query by hash. (2) Apollo Client normalized cache: caches by type+id on client side. (3) Server-side response caching: cache entire query result for anonymous/public data. (4) CDN caching via GET for persisted queries.

Q31. How do you rate-limit a GraphQL API?

Complexity analysis: assign cost to each field, reject queries exceeding cost threshold. Depth limiting: reject queries deeper than N levels. Amount limiting: cap list sizes (`users(first: N)` where N has max). Libraries: `graphql-query-complexity`. Combined with standard rate limiting at API gateway level.

Q32. What is the difference between @QueryMapping and @SchemaMapping in Spring GraphQL?

@QueryMapping: shortcut for @SchemaMapping(typeName='Query') — maps to a root Query field.
@SchemaMapping: maps a resolver to a specific field on a specific type. Example: @SchemaMapping(typeName='User', field='orders') maps to User.orders field. @BatchMapping is the DataLoader shortcut for @SchemaMapping with batch loading.

Q33. What is REST API versioning and what are the strategies?

URI: /api/v1/users (most common, explicit, breaks REST resource concept). Query param: /users?version=1 (messy). Header: Accept: application/vnd.myapp.v1+json (cleanest, less visible). Subdomain: v1.api.example.com. Best practice for large APIs: URI versioning for discoverability; maintain v1 until all consumers migrate.

Q34. How do you implement a GraphQL subscription in Spring Boot?

schema.graphqls: type Subscription { orderUpdated(orderId: ID!): Order }. Controller: @SubscriptionMapping public Flux orderUpdated(@Argument String orderId) { return orderUpdatePublisher.subscribe(orderId); }. Transport: WebSocket (spring.graphql.websocket.path=/graphql-ws) or HTTP SSE. Client connects via WebSocket and receives a stream of updates.

Q35. What is protoc and how is it used in a Java project?

protoc is the Protocol Buffer compiler. It reads .proto files and generates Java classes for messages and service stubs. In Maven/Gradle: protobuf-gradle-plugin auto-runs protoc on .proto files in src/main/proto/. Generates: message POJOs (UserProto.java), service base classes, and client stubs. Regenerate on every proto change—generated code is not committed to VCS.

Q36. Explain the differences between REST and RPC-style APIs.

REST: resource-oriented. URLs represent nouns (resources). HTTP verbs define actions. Stateless, cacheable, uniform interface. Example: GET /orders/123. RPC-style: action-oriented. URLs represent procedures/operations. Example: POST /getOrder with body {orderId: 123}. gRPC is explicitly RPC—service.Method() semantics, not resource manipulation.

Q37. How do you implement field-level authorization in REST vs GraphQL?

REST: return null or omit field based on security context in the DTO mapper. Or: use different DTO classes for different roles. GraphQL: field-level annotation (@PreAuthorize on @SchemaMapping methods), or custom directive @auth(requires: ADMIN). GraphQL's explicit field resolution makes per-field auth more natural than REST.

Q38. What is the trade-off of using a single GraphQL endpoint?

Advantage: no endpoint proliferation—all queries to /graphql. Client-driven data fetching. Disadvantage: no HTTP caching (POST to single endpoint). API gateway rate limiting is per-query not per-endpoint. Security scanning tools work less well with GraphQL than REST. Mitigations: persisted queries, cost analysis, Apollo Router for caching.

Q39. How do you handle large file uploads in REST vs gRPC vs GraphQL?

REST: multipart/form-data upload directly—well supported. gRPC: client-streaming RPC—send chunks as stream of bytes messages. GraphQL: awkward—spec doesn't define file upload. graphql-multipart-request-spec workaround, or use pre-signed URLs (S3) and pass URL as mutation argument.

Q40. What metrics should you track for API performance?

Latency: p50/p95/p99 response times. Throughput: requests/sec. Error rate: 4xx/5xx %. Payload size: average request/response size (important for mobile). For GraphQL: query complexity distribution, resolver

execution time. For gRPC: stream duration, messages per stream. Tools: Micrometer + Prometheus + Grafana.

Q41. What is the difference between a REST API and a RESTful API?

A REST API loosely follows REST conventions (uses HTTP, JSON, reasonable URLs). A truly RESTful API adheres to all 6 REST constraints including HATEOAS, statelessness, uniform interface. Most real-world 'REST APIs' are actually REST-ish or HTTP APIs—they don't implement HATEOAS and may have stateful behavior. Roy Fielding has criticized many 'REST APIs' for not being truly RESTful.

Q42. How does gRPC handle load balancing in Kubernetes?

HTTP/2 long-lived connections make L4 load balancing (kube-proxy) ineffective—all requests go to one pod. Solutions: (1) Client-side load balancing with service mesh (Istio/Linkerd intercepts at L7). (2) gRPC client-side load balancing with service discovery. (3) Headless Kubernetes Service returning all pod IPs. Most common: Istio.

Q43. What is schema stitching or federation in GraphQL?

Federation (Apollo): split GraphQL schema across multiple services. Each service owns its part of the schema. Apollo Router composes them into a unified supergraph. Services reference each other's types via @key directive. Enables team autonomy—each team owns their GraphQL subgraph without a monolithic schema.

Q44. Explain the REST uniform interface constraint.

The uniform interface constraint specifies: (1) Resource identification via URIs, (2) Resource manipulation through representations (JSON/XML body), (3) Self-descriptive messages (Content-Type, Accept headers), (4) HATEOAS—hypermedia links drive application state. This constraint decouples client from server implementation and enables independent evolution.

Q45. What is an API gateway and what cross-cutting concerns does it handle?

A reverse proxy sitting in front of all microservices. Handles: routing (route /api/orders to order service), authentication (validate JWT once), rate limiting, SSL termination, request logging, response caching, circuit breaking, protocol translation (REST to gRPC), request/response transformation. Options: Spring Cloud Gateway, Kong, AWS API Gateway, Nginx.

Q46. How do you implement streaming in REST?

Server-Sent Events (SSE): server pushes event stream to client over HTTP. Spring Boot: return SseEmitter or Flux. One-way (server to client). Suitable for notifications, live updates. WebSocket: bidirectional streaming. HTTP chunked transfer: send large response in chunks with Transfer-Encoding: chunked.

Q47. What is the OpenAPI contract-first approach?

Write the OpenAPI spec first before any code. Generate server stubs and client SDKs from the spec. Server implements the generated interface. This ensures API contract is the single source of truth. Contrast with code-first (annotate Spring controllers, generate spec from code). Contract-first is better for API-as-product where consumers start building before server is ready.

Q48. When should you use server-sent events vs WebSockets?

SSE: unidirectional server-to-client push. Simple, built on HTTP, auto-reconnects. Good for: live feeds, notifications, progress updates. Supports EventSource in browsers. WebSocket: bidirectional, lower latency, full-duplex. Good for: chat, collaborative editing, real-time games. GraphQL subscriptions can use either; SSE is preferred for simplicity when bidirectional isn't needed.

Q49. How do you document a gRPC API?

proto files ARE the documentation—service and message definitions are self-documenting. Add comments in proto files: they appear in generated code. Tools: protoc-gen-doc generates HTML/Markdown documentation from proto comments. buf.build hosts proto schemas publicly. Contrast: REST requires separate OpenAPI spec; GraphQL has built-in introspection.

Q50. What is REST API security best practice?

HTTPS always (TLS 1.2+). JWT Bearer tokens (short-lived, signed). OAuth2/OIDC for authorization. Rate limiting at gateway. Input validation (@Valid + Bean Validation). Output filtering (return only authorized fields). CORS configuration. Security headers (X-Content-Type-Options, X-Frame-Options). Never expose stack traces in error responses.